

Projeto 5 — UART por software entre dois Arduinos

Camadas Físicas da Computação · Insper · Explicação do código

1. O que o projeto faz

Dois Arduinos Uno trocam caracteres por um **pino digital comum** (pino 8 de cada um), sem usar o chip UART da placa. O transmissor (`tx.ino`) pega um byte e produz no pino, bit a bit e com o tempo certo, o frame do padrão UART: 1 start bit, 8 bits de dado, 1 bit de paridade par e 1 stop bit, a 9600 bits/s. O receptor (`rx.ino`) lê esse pino, detecta o start bit, amostra cada bit no meio da sua duração, remonta o byte, confere paridade e stop bit e imprime o resultado na serial USB — que é usada só para monitorar, como o enunciado permite.

Nenhum dos dois usa `delay()`: todo o tempo é controlado com `micros()`, e o `loop()` nunca fica parado esperando. Isso é o que atende o item “não utilizar função `delay` ou `sleep`” da rubrica.

```
Projeto 5/  
├ tx/tx.ino          transmissor: envia "Camadas Físicas " em loop, 1 caractere a cada 250 ms  
├ tx/sketch.yaml     placa e porta padrao pro arduino-cli (Uno, COM5)  
├ rx/rx.ino          receptor: maquina de estados, amostra no meio do bit, acusa erro de paridade e de frame  
├ rx/sketch.yaml     idem (Uno, COM8)  
├ teste/Arduino.h    imitacao minima do Arduino, pra rodar os dois .ino no PC  
├ teste/teste.cpp     simulacao: liga tx e rx por um fio virtual e confere 5 cenarios  
└ README.md          montagem, comandos e roteiro da apresentacao
```

2. O frame UART

A linha fica em **1** quando não há nada sendo transmitido (*idle*). Um frame começa quando ela cai para 0 (start bit), seguem os 8 bits do dado **do menos significativo para o mais significativo** (LSB primeiro), depois o bit de paridade e por fim o stop bit em 1, que devolve a linha ao repouso. Cada bit dura $T = 1/\text{baud}$; a 9600 baud, **$T = 104 \mu\text{s}$** . Um frame completo tem 11 bits, ou 1,14 ms.

Exemplo com o caractere '**C**' (ASCII 0x43 = 0100 0011). Enviado LSB primeiro, os bits de dado saem na ordem 1 1 0 0 0 0 1 0. Há três 1s (ímpar), então o bit de paridade par vale **1** para o total ficar par.

bit no fio	start	d0	d1	d2	d3	d4	d5	d6	d7	par	stop
nível	0	1	1	0	0	0	0	1	0	1	1
instante (×T)	0	1	2	3	4	5	6	7	8	9	10

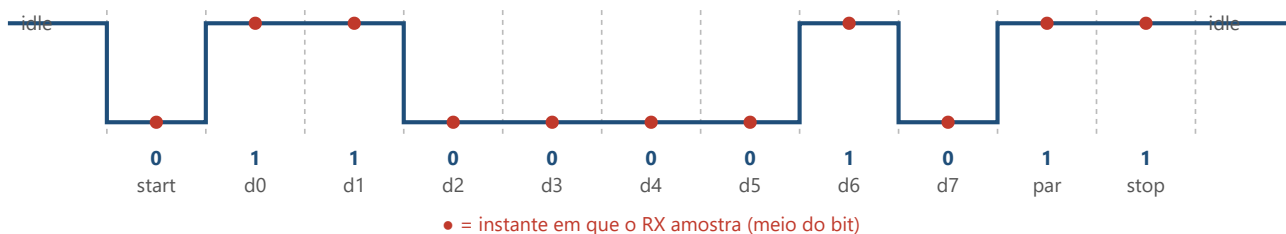


Figura 1 — o frame de 'C' no pino, com os pontos onde o receptor lê cada bit.

Paridade

Paridade **par** significa: dado + bit de paridade têm um número par de 1s. O transmissor conta os 1s do byte e põe 1 no bit de paridade quando a contagem é ímpar. O receptor faz a mesma conta sobre o byte que remontou e compara com o bit que chegou; se diferem, algum bit virou no caminho. Detecta qualquer erro de **1 bit** (ou número ímpar de bits) — é a versão de 1 bit do CRC-16 usado no Projeto 4. A função é idêntica nos dois sketches:

```
uint8_t paridadePar(uint8_t dado) {
    uint8_t uns = 0;
    for (uint8_t i = 0; i < 8; i++) uns += (dado >> i) & 0x01;    // isola o bit i e soma
    return uns % 2;                                                // 1 se a quantidade de 1s for impar
}
```

3. Decisões de projeto

Decisão	Por quê
Tempo por <code>micros()</code> , nunca <code>delay()</code>	O <code>loop()</code> roda livre e a cada volta pergunta “já é hora do próximo bit?”. O Arduino continua respondendo (serial, teclado) e a rúbrica exige.
Horário de cada bit calculado a partir do início do frame	<code>tInicio + i·T</code> em vez de “último bit + T”: o atraso de uma volta do loop não se acumula nos bits seguintes.
Receptor amostra no meio do bit	Na borda o nível está mudando e a leitura é imprevisível. No meio há T/2 de folga para cada lado.
Frame guardado como um inteiro de 11 bits	O bit <i>i</i> do frame é <code>(frame >> i) & 1</code> : o loop do TX vira 3 linhas.
Relógio do RX é a borda do start bit	Não existe clock compartilhado; o start bit é a única referência de tempo. Por isso ele é estimado com cuidado (seção 5.2).
Serial USB a 115200, só para monitorar	Escoa rápido e não participa da transmissão. O enunciado libera esse uso.

4. tx/tx.ino — o transmissor

4.1 Constantes

```
const uint8_t PINO_TX      = 8;
const uint32_t BAUD        = 9600;
const uint32_t T_BIT       = 1000000UL / BAUD;    // 104 us a 9600
const uint32_t INTERVALO   = 250000UL;           // us de silencio entre frames (so pra dar tempo de ler o monitor)
const char     MENSAGEM[]  = "Camadas Físicas ";
uint8_t        ERRO_A_CADA = 0;    // DEMO do erro de paridade: inverte a paridade a cada N frames (0 = desligado)
const uint8_t  BITS_FRAME  = 11; // 1 start + 8 dados + 1 paridade + 1 stop
```

`INTERVALO` existe só para a demonstração ficar legível (4 caracteres por segundo). O mínimo seguro é ~10 ms, porque a linha que o RX imprime leva ~6 ms para escoar pela USB e, se o próximo start chegasse antes, seria perdido. `ERRO_A_CADA` é o botão da demo do item “identificação de erro de paridade”: com 5, um em cada cinco frames sai com a paridade errada de propósito.

4.2 Montagem do frame

```
uint16_t montaFrame(uint8_t dado, bool inverteParidade) {
    uint8_t p = paridadePar(dado) ^ (inverteParidade ? 1 : 0);
    // bit 0 = start (0) | bits 1..8 = dado | bit 9 = paridade | bit 10 = stop (1)
    return (1u << 10) | ((uint16_t)p << 9) | ((uint16_t)dado << 1);
}
```

O frame inteiro cabe num `uint16_t`. O dado é deslocado uma posição para a esquerda para abrir espaço ao start bit, que fica 0 sozinho no bit 0. Como o TX emite do bit 0 ao bit 10, o d0 do dado (bit 1 do frame) sai logo depois do start: LSB primeiro, como o padrão manda.

4.3 O `loop()` em duas fases

```
void loop() {
    uint32_t agora = micros();

    if (bitAtual < BITS_FRAME) { // FASE 1: no meio de um frame
        if (agora - tInicio >= (uint32_t)bitAtual * T_BIT) {
            digitalWrite(PINO_TX, (frame >> bitAtual) & 0x01); // põe o bit i no pino
            bitAtual++;
        }
        return;
    }

    if (agora - tInicio >= (uint32_t)BITS_FRAME * T_BIT + INTERVALO) { // FASE 2: intervalo acabou
        uint8_t dado = MENSAGEM[idx];
        idx = (idx + 1) % (sizeof(MENSAGEM) - 1);
        enviados++;
        bool erro = ERRO_A_CADA && (enviados % ERRO_A_CADA == 0);
        frame = montaFrame(dado, erro);
        Serial.print(...); // mostra o que vai sair
        bitAtual = 0;
        tInicio = micros(); // depois do print, pra ele não roubar tempo do start bit
    }
}
```

- **Fase 1** (`bitAtual < 11`): o horário do bit i é $tInicio + i \cdot T$. Quando chega, escreve $(frame \gg i) \& 1$ no pino e avança. Uma volta do loop leva uns 8 μs , então cada borda sai no máximo 8 μs atrasada — e esse atraso *não* se propaga ao bit seguinte.
- **Fase 2** (`bitAtual == 11`): passados $11T + INTERVALO$ desde o início do frame anterior, escolhe o próximo caractere, monta o frame e zera `bitAtual`. O `tInicio` é lido *depois* do `Serial.print`: o print gasta ~50 μs e, se viesse depois de marcar o tempo, encurtaria o start bit pela metade.
- Toda comparação de tempo é `agora - tInicio >= ...` em aritmética *unsigned*: continua correta quando `micros()` dá a volta (a cada 71 minutos).

5. rx/rx.ino — o receptor

5.1 Máquina de estados

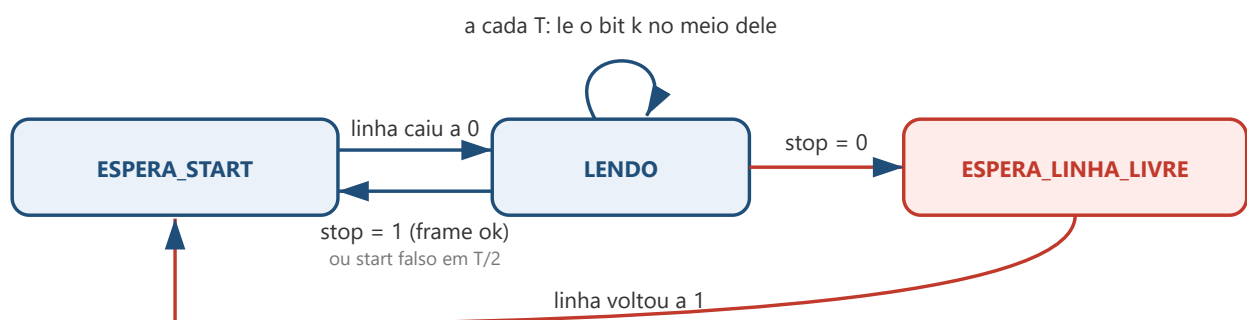


Figura 2 — os três estados do receptor.

O `loop()` é um `switch` (estado). Variáveis globais guardam o que está sendo montado: `tStart` (quando o frame começou), `bitAtual` (qual bit é o próximo: 0 = start, 1..8 = dados, 9 = paridade, 10 = stop), `dado` (o byte sendo remontado) e `paridadeRx` (o bit de paridade lido).

5.2 ESPERA_START: detectar o start e estimar quando ele começou

```
case ESPERA_START:
    if (digitalRead(PINO_RX) == HIGH) {
        tUltimoHigh = agora;          // linha em repouso: anota a hora
    } else {                          // borda de descida = começo do start bit
        uint32_t gap = agora - tUltimoHigh; // se o loop travou (print bloqueou), o ponto medio nao vale: usa agora
        tStart = agora - (gap < T_BIT ? gap / 2 : 0);
        bitAtual = 0;
        dado = 0;
        estado = LENDO;
    }
    break;
```

Aqui está o detalhe mais importante do receptor. O RX só percebe que a linha caiu *na volta seguinte* do loop, então a borda real aconteceu em algum ponto entre a última leitura HIGH e esta leitura LOW. Se usássemos `tStart = agora`, o relógio do frame ficaria sempre atrasado e todos os pontos de amostragem deslocariam para a direita, comendo a folga do lado do stop bit. Estimar a borda no **meio do intervalo** entre as duas leituras zera esse viés. Foi isso que fez o teste com cristal 3% fora passar (seção 7).

O `gap < T_BIT` é uma proteção: se o loop ficou travado entre as duas leituras (por exemplo, o `Serial.print` bloqueou com o buffer cheio), o “meio” seria um instante qualquer no passado e o frame sairia lixo; nesse caso cai-se para `tStart = agora`. A função `voltaAoRepouso(agora)` faz `estado = ESPERA_START`; `tUltimoHigh = agora`; e é chamada em toda transição de volta a esse estado — sempre logo após uma leitura HIGH — para `tUltimoHigh` nunca ficar velho.

5.3 LENDO: amostrar cada bit no meio

```
case LENDO:
    if (agora - tStart >= (uint32_t)bitAtual * T_BIT + T_BIT / 2) { // meio do bit k
        uint8_t nivel = digitalRead(PINO_RX);

        if (bitAtual == 0) { // confirma o start no meio dele: filtra ruído
            if (nivel == HIGH) voltaAoRepouso(agora);
        }
        else if (bitAtual <= 8) { // dados, LSB primeiro
            dado |= nivel << (bitAtual - 1);
        }
        else if (bitAtual == 9) {
            paridadeRx = nivel;
        }
        else { // bit 10 = stop, fecha o frame
            concluiFrame(nivel);
            if (nivel == HIGH) voltaAoRepouso(agora); else estado = ESPERA_LINHA_LIVRE;
        }
        bitAtual++;
    }
    break;
```

O bit k é lido em $tStart + k \cdot T + T/2$. O primeiro ($k = 0$) é o próprio start bit, relido no meio: se voltou a HIGH, foi um ruído curto e não um frame — volta a esperar sem contar nada. Os bits 1..8 são o dado: o bit que chega primeiro é o d0, então cada nível lido é deslocado para a posição `bitAtual - 1` e somado ao byte com `|=` — o inverso exato do `>>` e `&` do transmissor. O bit 9 é a paridade e o 10 o stop, que fecha o frame.

5.4 Fechar o frame e classificar

```
void concluiFrame(uint8_t stopBit) {
    const char* status;
    if (stopBit != HIGH) { errosFrame++; status = "ERRO DE FRAME (stop bit = 0)"; }
    else if (paridadeRx != paridadePar(dado)) { errosParidade++; status = "ERRO DE PARIDADE"; }
    else { recebidosOk++; status = "OK"; }
    Serial.print(...); // <- 'C' 0x43 paridade=1 OK [ok 12 | paridade 0 | frame 0]
}
```

A ordem importa: se o stop bit não é 1, os bits nem estavam alinhados e a paridade não significa nada, então é erro de *frame*. Com o frame íntegro, a paridade recalculada é comparada com a recebida. Os três contadores aparecem em toda linha do monitor, o que facilita mostrar a detecção na apresentação.

5.5 ESPERA_LINHA_LIVRE: recuperar de um frame quebrado

Só se entra aqui após um erro de frame. Se a linha está presa em 0 (fio no GND, desalinhamento), sem esse estado o próximo 0 lido seria tomado como um novo start e o RX cuspiria frames de lixo em sequência. Ele espera a linha voltar a 1 antes de aceitar outro start — o mesmo papel da resincronização por EOP do Projeto 4.

5.6 Detalhes

- `pinMode(PINO_RX, INPUT_PULLUP)`: com o fio solto o pino lê HIGH (repouso) em vez de flutuar e gerar frames fantasmas.
- `Serial.begin(115200)` no RX: escoa a linha de log em ~6 ms, bem dentro do intervalo de 250 ms entre frames.

6. Resumo do tratamento de erros

Situação	Como o RX percebe	O que faz
Um bit de dado virou	Paridade recalculada ≠ recebida	Imprime <code>ERRO DE PARIDADE</code> , conta em <code>errosParidade</code>
Bits desalinhados / linha presa em 0	Stop bit lido como 0	Imprime <code>ERRO DE FRAME</code> , vai para <code>ESPERA_LINHA_LIVRE</code> até a linha subir
Pulso curto de ruído no repouso	Start relido em $T/2$ já está em 1	Descarta em silêncio, volta a <code>ESPERA_START</code>
Loop travou antes de um start	<code>gap >= T_BIT</code>	Ignora o ponto médio, usa <code>tStart = agora</code>
Fio desconectado	Pull-up mantém a linha em 1	Fica em <code>ESPERA_START</code> , nada é impresso

7. Teste em simulação (sem Arduino)

Antes de gravar nas placas, os dois sketches foram executados **no PC**, ligados por um fio virtual. A ideia é compilar os mesmos arquivos `.ino`, sem nenhuma alteração, contra uma imitação mínima do Arduino.

7.1 teste/Arduino.h — o Arduino de mentira

Implementa apenas o que os sketches usam, em cima de três variáveis globais:

Variável	Papel
<code>g_micros</code>	O relógio simulado. <code>micros()</code> devolve-o arredondado para múltiplo de 4 μ s, como no Uno real.
<code>g_fio</code>	O nível lógico do fio. <code>digitalWrite</code> escreve nele, <code>digitalRead</code> lê dele: é o jumper entre as placas.
<code>g_escala</code>	Multiplica o tempo que um lado enxerga. <code>1.03</code> = “este Arduino acha que o tempo passa 3% mais rápido” = cristal fora do valor.
<code>SerialMock</code>	Acumula os <code>Serial.print</code> numa <code>std::string</code> em vez de mandar para a USB.

7.2 teste/teste.cpp — o cenário

```
namespace TX { #include "../tx/tx.ino" } // cada sketch dentro do seu namespace:
namespace RX { #include "../rx/rx.ino" } // setup()/loop() e globais nao colidem

// dois processadores independentes: cada um roda seu loop() a cada 4..8 us
static void passo(double escalaRx) {
    g_micros++;
    if (g_micros >= tProxTX) { g_escala = 1.0; TX::loop(); tProxTX = g_micros + 4 + rng() % 5; }
    if (g_micros >= tProxRX) { g_escala = escalaRx; RX::loop(); tProxRX = g_micros + 4 + rng() % 5; }
}
```

Os `.ino` são incluídos como texto, cada um dentro de um `namespace`, então o teste acessa `TX::enviados`, `RX::recebidosOk`, `RX::estado` diretamente. `passo()` avança o mundo 1 μ s e dá a cada “processador” a sua volta de `loop()` quando chega a hora dele, com duração sorteada entre 4 e 8 μ s — o custo real de `micros()` + `digitalRead/Write` num ATmega328. A função `roda(n)` repete `passo()` até o TX ter enviado *n* frames e registra o byte de cada frame que o RX fecha.

7.3 Cenários e resultado

#	Cenário	Confere
1	20 frames limpos	20 OK, 0 erros, bytes idênticos a “Camadas Físicas Cama”
2	<code>TX::ERRO_A_CADA = 5</code>	Exatamente 4 erros de paridade em 20; os bytes continuam certos (só a paridade foi mexida)
3	RX com relógio 3% rápido e depois 3% lento	Decodifica tudo. O real é $\leq 1\%$ (ressonador do Uno), então há folga
4a	Pulso LOW de 20 μ s no repouso	Nenhum frame contado; RX volta a <code>ESPERA_START</code>
4b	Linha presa em 0 por 15 bits (break)	1 <code>ERRO DE FRAME</code> , entra em <code>ESPERA_LINHA_LIVRE</code> ; ao soltar a linha, recebe os 3 frames seguintes normalmente

```
$ g++ -std=c++17 -I teste teste/teste.cpp -o teste/teste.exe && teste/teste.exe
ok: 20 frames limpos, bytes identicos
ok: 4 erros de paridade detectados, bytes ainda identicos
ok: RX com cristal +-3% ainda decodifica tudo
ok: glitch curto ignorado
ok: break gerou ERRO DE FRAME e o RX voltou a receber

ultimas linhas do monitor serial (TX e RX misturados):
<- 'a' 0x61 paridade=1 OK [ok 78 | paridade 4 | frame 1]
-> 'm' 0x6D paridade=1
<- 'm' 0x6D paridade=1 OK [ok 79 | paridade 4 | frame 1]

=== todos os cenarios passaram ===
```

O que o teste pegou. Na primeira versão, `tStart = agora`. Com o RX 3% lento, a amostra do bit de paridade caía em $\sim 9,9T$ — em cima da borda do stop bit — e 6 dos 20 frames davam erro. A estimativa pelo ponto médio (seção 5.2) resolveu. Em seguida o teste mostrou que um salto no relógio entre duas leituras produzia um frame de lixo, o que levou ao `guard gap < T_BIT`.

8. Como compilar, gravar e apresentar

8.1 Montagem

- **Pino 8** do TX → **pino 8** do RX; **GND** com **GND**.
- Reset **livre** nas duas placas (agora o processador executa o código, ao contrário dos projetos anteriores).
- Analog Discovery: um canal digital no pino 8 do TX e o GND.

8.2 Comandos

```
arduino-cli compile tx ; arduino-cli upload tx      # porta em tx/sketch.yaml (COM5)
arduino-cli compile rx ; arduino-cli upload rx      # porta em rx/sketch.yaml (COM8)
arduino-cli monitor -p COM8 -c baudrate=115200     # monitor do RX
```

Saída esperada no RX: `<- 'C' 0x43 paridade=1 OK [ok 12 | paridade 0 | frame 0]`

8.3 Roteiro por item da rubrica

Item	Requisito	Como mostrar
C+	≥ 9600 baud	BAUD = 9600 nos dois sketches; o RX imprime os caracteres corretos de “Camadas Físicas”.
B+	Identificação de erro de paridade	Em <code>tx.ino</code> , <code>ERRO_A_CADA = 5</code> e regravar: a cada 5 frames o RX imprime <code>ERRO DE PARIDADE</code> e o contador sobe; o TX avisa “INVERTIDA de proposito”.
A	Sem delay/sleep	Todo o tempo é por <code>micros()</code> ; não há nenhuma chamada a <code>delay</code> nos dois arquivos.
A+	Monitorar com Analog Discovery	WaveForms → Logic Analyzer → Add → UART : canal do pino, 9600 baud, 8 bits, parity Even , 1 stop, idle high . Base de tempo $\sim 200 \mu s/div$ mostra o frame inteiro e o decodificador exibe o caractere.

9. Limites conhecidos e caminho de evolução

- **Baud máximo ~ 19200 .** `micros()` tem passo de $4 \mu s$ e `digitalWrite/digitalRead` levam $\sim 4 \mu s$ cada; a 38400 baud ($T = 26 \mu s$) o jitter já é comparável ao bit. Para ir além: no TX, Timer1 em modo CTC disparando cada bit e escrita direta em `PORTB`; no RX, `attachInterrupt` na borda do start bit e leitura direta de `PINB`.

- **Intervalo mínimo entre frames ~10 ms**, por causa do log do RX na USB. Para transmitir contínuo, encurtar a linha impressa ou imprimir só contadores a cada N frames.
- **Tolerância de relógio $\pm 3\%$** verificada em simulação; o hardware real fica em $\pm 1\%$.

Apêndice A — tx/tx.ino completo

```
// =====
// TX - UART por software num pino digital generico (Projeto 5)
//
// Frame: idle(1) | start(0) | d0 d1 d2 d3 d4 d5 d6 d7 (LSB primeiro) | paridade PAR | stop(1)
//
// Sem delay(): cada bit e agendado por micros() a partir do INICIO do frame
// (tInicio + i*T), entao o erro de tempo de um bit nao acumula no seguinte.
// Ligacao: pino 8 daqui -> pino 8 do RX, e GND com GND. Reset LIVRE.
// =====

#include <Arduino.h>

const uint8_t PINO_TX      = 8;
const uint32_t BAUD        = 9600;
const uint32_t T_BIT       = 1000000UL / BAUD;    // 104 us a 9600
const uint32_t INTERVALO   = 250000UL;           // us de silencio entre frames, so pra dar tempo de ler o monitor
                                                    // (minimo ~10 ms: o print do RX leva ~6 ms a 115200 e engoliria o proximo

start()
const char    MENSAGEM[] = "Camadas Fisicas ";
uint8_t       ERRO_A_CADA = 0;    // DEMO do erro de paridade: inverte o bit de paridade a cada N frames (0 = desligado)

const uint8_t BITS_FRAME = 11; // 1 start + 8 dados + 1 paridade + 1 stop

// ponytail: micros() tem passo de 4 us e digitalWrite leva ~4 us, entao isso vale
// ate ~19200 baud. Acima disso: timer1 em modo CTC + escrita direta no PORTB.

uint8_t paridadePar(uint8_t dado) {
    uint8_t uns = 0;
    for (uint8_t i = 0; i < 8; i++) uns += (dado >> i) & 0x01;
    return uns % 2;    // paridade PAR: bit = 1 se o numero de 1s for impar, pra ficar par no total
}

uint16_t montaFrame(uint8_t dado, bool inverteParidade) {
    uint8_t p = paridadePar(dado) ^ (inverteParidade ? 1 : 0);
    // bit 0 = start (0) | bits 1..8 = dado | bit 9 = paridade | bit 10 = stop (1)
    return (1u << 10) | ((uint16_t)p << 9) | ((uint16_t)dado << 1);
}

uint16_t frame;
uint8_t bitAtual = BITS_FRAME;    // == BITS_FRAME: frame terminou, estamos no intervalo
uint32_t tInicio;
uint16_t enviados = 0;
uint8_t idx = 0;

void setup() {
    pinMode(PINO_TX, OUTPUT);
    digitalWrite(PINO_TX, HIGH);    // linha em repouso
    Serial.begin(115200);            // USB: so pra monitorar, nao participa da transmissao
    Serial.print("TX UART por software: "); Serial.print(BAUD); Serial.println(" baud, 8 dados, paridade PAR, 1 stop");
    tInicio = micros();
}
```

```

void loop() {
    uint32_t agora = micros();

    if (bitAtual < BITS_FRAME) {
        // a subtracao unsigned continua certa quando micros() da a volta (a cada ~70 min)
        if (agora - tInicio >= (uint32_t)bitAtual * T_BIT) {
            digitalWrite(PINO_TX, (frame >> bitAtual) & 0x01);
            bitAtual++;
        }
        return;
    }

    if (agora - tInicio >= (uint32_t)BITS_FRAME * T_BIT + INTERVALO) {
        uint8_t dado = MENSAGEM[idx];
        idx = (idx + 1) % (sizeof(MENSAGEM) - 1);
        enviados++;
        bool erro = ERRO_A_CADA && (enviados % ERRO_A_CADA == 0);
        frame = montaFrame(dado, erro);

        Serial.print("-> "); Serial.print((char)dado); Serial.print("' 0x"); Serial.print(dado, HEX);
        Serial.print(" paridade="); Serial.print((frame >> 9) & 0x01);
        if (erro) Serial.print(" (INVERTIDA de proposito)");
        Serial.println();

        bitAtual = 0;
        tInicio = micros(); // depois do print, pra ele nao roubar tempo do start bit
    }
}

```

Apêndice B — `rx/rx.ino` completo

```
// =====
// RX - recebe um frame UART num pino digital generico, sem delay() (Projeto 5)
//
// Maquina de estados:
// ESPERA_START ---(linha caiu pra 0)---> LENDO ---(stop bit lido)---> ESPERA_START
// \---(stop bit = 0)---> ESPERA_LINHA_LIVRE
//
// O relógio do frame e o instante da borda de descida do start bit (tStart).
// O bit k e amostrado no MEIO dele: tStart + k*T + T/2 - nunca na transicao.
// Como so percebemos a borda na volta seguinte do loop(), ela caiu em algum ponto
// entre a ultima leitura HIGH e a primeira LOW: tStart = meio desse intervalo.
// Isso tira o atraso sistematico e sobra folga pra cristais fora do valor.
// Ligacao: pino 8 do TX -> pino 8 daqui, GND com GND. Monitor serial a 115200.
// =====
#include <Arduino.h>

const uint8_t PINO_RX = 8;
const uint32_t BAUD = 9600;
const uint32_t T_BIT = 1000000UL / BAUD; // 104 us a 9600

// ponytail: polling com digitalRead (~4 us) + micros() com passo de 4 us aguenta ate
// ~19200 baud. Acima disso: attachInterrupt no start bit e leitura direta do PINB.

enum Estado { ESPERA_START, LENDO, ESPERA_LINHA_LIVRE };

Estado estado = ESPERA_START;
uint32_t tStart;
uint32_t tUltimoHigh; // ultima vez que a linha foi lida em repouso (HIGH)
uint8_t bitAtual; // 0 = start, 1..8 = dados, 9 = paridade, 10 = stop
uint8_t dado;
uint8_t paridadeRx;

uint16_t recebidosOk = 0, errosParidade = 0, errosFrame = 0;

void voltaAoRepouso(uint32_t agora) { // sempre chamado logo apos ler a linha em HIGH
    estado = ESPERA_START;
    tUltimoHigh = agora;
}

uint8_t paridadePar(uint8_t d) {
    uint8_t uns = 0;
    for (uint8_t i = 0; i < 8; i++) uns += (d >> i) & 0x01;
    return uns % 2;
}

void concluiFrame(uint8_t stopBit) {
    const char* status;
    if (stopBit != HIGH) { errosFrame++; status = "ERRO DE FRAME (stop bit = 0)"; }
    else if (paridadeRx != paridadePar(dado)) { errosParidade++; status = "ERRO DE PARIDADE"; }
    else { recebidosOk++; status = "OK"; }
}
```

```

Serial.print("<- "); Serial.print((char)dado); Serial.print("' 0x"); Serial.print(dado, HEX);
Serial.print(" paridade="); Serial.print(paridadeRx);
Serial.print(" "); Serial.print(status);
Serial.print(" [ok "); Serial.print(recebidosOk);
Serial.print(" | paridade "); Serial.print(errosParidade);
Serial.print(" | frame "); Serial.print(errosFrame); Serial.println("]");
}

void setup() {
  pinMode(PINO_RX, INPUT_PULLUP); // fio solto le HIGH (= idle) em vez de lixo
  Serial.begin(115200);
  Serial.print("RX UART por software: "); Serial.print(BAUD); Serial.println(" baud, 8 dados, paridade PAR, 1 stop");
  tUltimoHigh = micros();
}

void loop() {
  uint32_t agora = micros();

  switch (estado) {

    case ESPERA_START:
      if (digitalRead(PINO_RX) == HIGH) {
        tUltimoHigh = agora;
      } else {
        // borda de descida = comeco do start bit
        uint32_t gap = agora - tUltimoHigh; // se o loop travou (print bloqueou), o ponto medio nao vale: usa agora
        tStart = agora - (gap < T_BIT ? gap / 2 : 0);
        bitAtual = 0;
        dado = 0;
        estado = LENDO;
      }
      break;

    case LENDO:
      if (agora - tStart >= (uint32_t)bitAtual * T_BIT + T_BIT / 2) { // meio do bit k
        uint8_t nivel = digitalRead(PINO_RX);

        if (bitAtual == 0) {
          // confirma o start no meio dele: filtra ruido
          if (nivel == HIGH) voltaAoRepouso(agora);
        }
        else if (bitAtual <= 8) {
          // dados, LSB primeiro
          dado |= nivel << (bitAtual - 1);
        }
        else if (bitAtual == 9) {
          paridadeRx = nivel;
        }
        else {
          // bit 10 = stop, fecha o frame
          concluiFrame(nivel);
          if (nivel == HIGH) voltaAoRepouso(agora); else estado = ESPERA_LINHA_LIVRE;
        }
        bitAtual++;
      }
      break;

    case ESPERA_LINHA_LIVRE: // stop bit errado: espera a linha voltar a 1 pra nao confundir dado com start
      if (digitalRead(PINO_RX) == HIGH) voltaAoRepouso(agora);
  }
}

```

```
break;
```

```
}
```

```
}
```